PROJECT PLAN

CodeShelf

A simple online bookstore for programming books

Document	Project Plan, version 1.0
Date	2 October 2026
Project duration	5 October 2026 - 18 December 2026 (11 weeks)
Methodology	Agile (Scrum-lite), one-week sprints
Project team	Georgiy Yeremin (78134) Uladzislau Yeudakimau (78057) Ruslana Savich (78169)

Contents

1. Project Overview
2. Scope
3. Team and Roles
4. Requirements
5. Technology Stack and Architecture
6. Work Breakdown Structure
7. Schedule (Gantt Chart)
8. Milestones and Deliverables
9. Responsibility Matrix (RACI)
10. Risk Management
11. Communication Plan
12. Quality Assurance and Testing
13. Acceptance Criteria

1. Project Overview

CodeShelf is a simple web-based online store that sells books about programming and software engineering. Customers can browse a catalog organised by programming language, topic and difficulty level, search for titles, add books to a shopping cart and place orders. Store staff manage the catalog and orders through an administration panel.

1.1 Goals

- Deliver a working, deployed online bookstore (MVP) within 11 weeks.
- Provide a clear, fast and mobile-friendly shopping experience for developers and IT students.
- Make it easy to find books by language (Python, Java, JavaScript, C#, C++, Go, etc.), topic and level.
- Give administrators simple tools to manage books, stock and orders.
- Practise a structured team workflow: planning, version control, code review, testing and deployment.

1.2 Target Users

- **Customers** - students, junior and senior developers looking for programming literature.
- **Administrator** - store staff who maintain the catalog and process orders.

2. Scope

In scope	Out of scope
Book catalog with categories (language, topic, level) Search, filtering and sorting Book details page (description, author, price, ISBN, cover) User registration, login and profile Shopping cart and checkout Test-mode payment (Stripe sandbox) or cash on delivery Order history for customers Admin panel: books, categories, stock, orders Responsive web design (desktop and mobile) Deployment to a public test server	Native mobile applications E-books / digital downloads Real payment processing with live money Multi-vendor marketplace features Recommendation engine / machine learning Multi-language interface (English only) Integration with real shipping carriers Loyalty programmes and gift cards

3. Team and Roles

Team member	Role	Main responsibilities
Georgiy Yeremin (78134)	Project Manager, Backend Developer	Project planning and coordination, sprint planning, progress tracking and reporting; REST API for users, authentication (JWT), cart and orders; code review; final presentation.
Uladzislau Yeudakimau (78057)	Backend and Database Developer, DevOps	Database design (PostgreSQL, ER model), catalog and search API, admin panel API; Docker setup, CI/CD pipeline, deployment and server configuration.
Ruslana Savich (78169)	Frontend Developer, UI/UX Designer, QA Lead	Wireframes and visual design, React user interface (catalog, book page, cart, checkout, profile, admin screens); test plan, manual and automated testing, bug tracking.

All team members take part in requirements analysis, code reviews, testing and documentation.

4. Requirements

4.1 Functional Requirements

ID	Requirement	Priority
FR-01	Customer can browse the book catalog with pagination.	Must
FR-02	Customer can filter books by programming language, topic and level, and sort by price, title or date.	Must
FR-03	Customer can search books by title, author or ISBN.	Must
FR-04	Customer can view a book details page (cover, description, author, publisher, year, pages, price, stock).	Must
FR-05	Customer can register, log in, log out and edit their profile.	Must
FR-06	Customer can add, remove and change the quantity of books in the cart.	Must
FR-07	Customer can place an order with a delivery address and a payment method (test payment or cash on delivery).	Must
FR-08	Customer receives an order confirmation (on screen and by e-mail).	Should
FR-09	Customer can view their order history and order status.	Should
FR-10	Administrator can create, edit and delete books and categories, and upload cover images.	Must
FR-11	Administrator can manage stock levels and change order status.	Must
FR-12	Customer can leave a rating and a short review for a purchased book.	Could
FR-13	Customer can add books to a wishlist.	Could

Priorities follow the MoSCoW method: Must, Should, Could. "Could" items are delivered only if time allows.

4.2 Non-functional Requirements

- **Performance:** catalog and book pages load in under 2 seconds on a standard connection.
- **Security:** passwords hashed with bcrypt, HTTPS, JWT-based sessions, input validation, protection against SQL injection and XSS, role-based access to the admin panel.
- **Usability:** responsive layout for screens from 360 px wide; checkout in no more than 3 steps.
- **Reliability:** daily database backups on the test server.
- **Maintainability:** consistent code style (ESLint, Prettier), README and API documentation.
- **Compatibility:** latest versions of Chrome, Firefox, Edge and Safari.

5. Technology Stack and Architecture

Layer	Technology
Frontend	React with TypeScript, Vite, React Router, Tailwind CSS
Backend	Node.js with Express, REST API, JWT authentication
Database	PostgreSQL with Prisma ORM
Payments	Stripe in test (sandbox) mode; cash on delivery option
Testing	Jest, React Testing Library, Supertest (API), Playwright (end-to-end)
DevOps	Git and GitHub, GitHub Actions (CI), Docker, deployment to Render or a VPS
Project management	GitHub Projects (Kanban board), weekly meetings, team chat
Design	Figma (wireframes and UI mockups)

5.1 Architecture

The system uses a classic three-tier architecture. The React single-page application communicates with the Express REST API over HTTPS using JSON. The API contains the business logic (catalog, cart, orders, authentication, administration) and stores data in PostgreSQL. Book cover images are stored on the server file system or object storage.

5.2 Main Data Entities

Entity	Key attributes
User	id, name, email, password_hash, role (customer / admin), created_at
Book	id, title, author, isbn, publisher, year, pages, description, price, stock, cover_url, level
Category	id, name, type (language / topic)
Cart / CartItem	user_id, book_id, quantity
Order / OrderItem	id, user_id, status, total, address, payment_method, created_at; book_id, quantity, price
Review	id, user_id, book_id, rating, comment

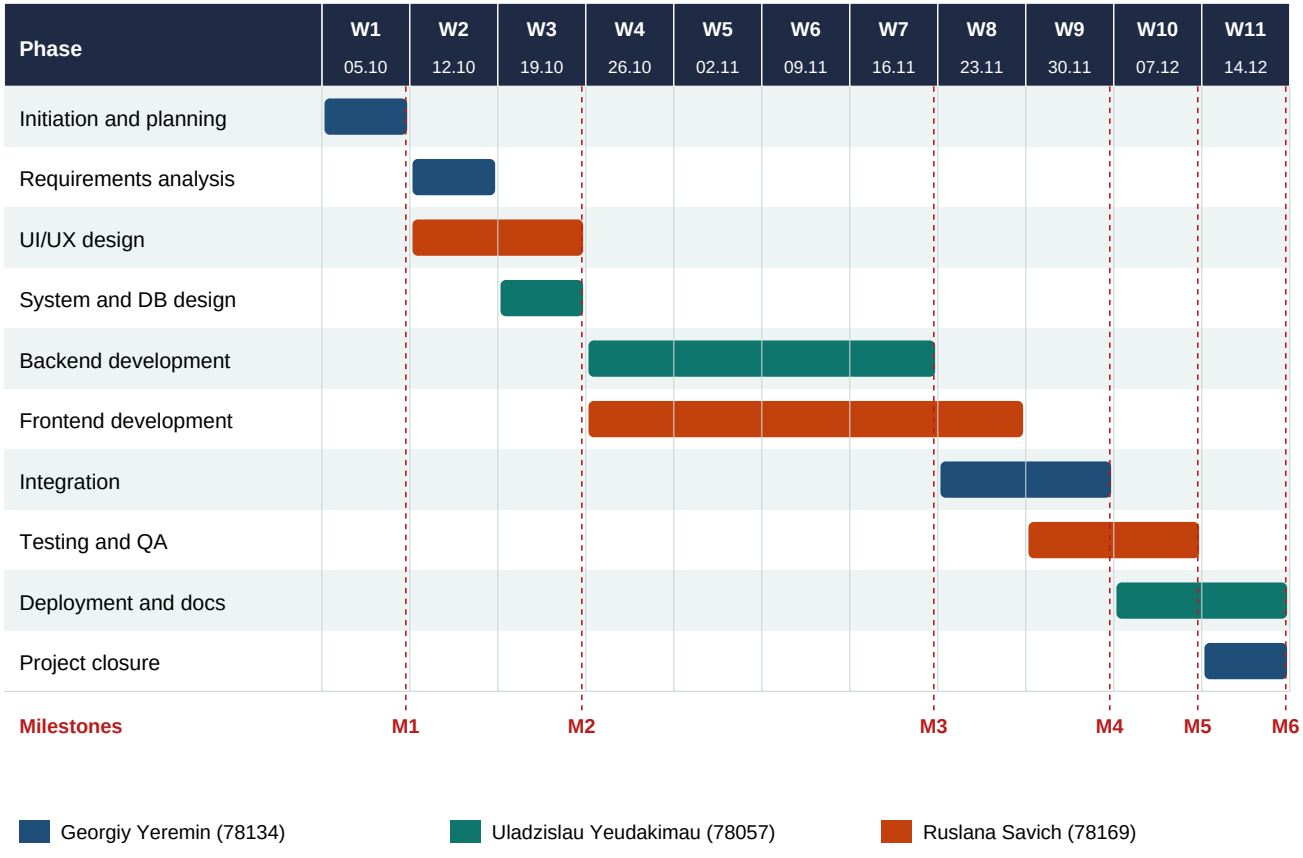
6. Work Breakdown Structure

#	Phase / work package	Lead	Dates	Effort (h)
1	**Initiation and planning**: project charter, plan, repository, Kanban board	Georgiy Yeremin (78134)	05.10 - 09.10	20
2	**Requirements analysis**: user stories, use cases, acceptance criteria	Georgiy Yeremin (78134)	12.10 - 16.10	24
3	**UI/UX design**: wireframes, mockups, style guide	Ruslana Savich (78169)	12.10 - 23.10	30
4	**System and database design**: architecture, ER diagram, API specification	Uladzislau Yeudakimau (78057)	19.10 - 23.10	24
5	**Backend development**: auth, catalog, search, cart, orders, admin API	Uladzislau Yeudakimau (78057) / Georgiy Yeremin (78134)	26.10 - 20.11	130
6	**Frontend development**: all customer and admin screens	Ruslana Savich (78169)	26.10 - 27.11	120
7	**Integration**: connecting frontend and API, payment sandbox, e-mails	Georgiy Yeremin (78134)	23.11 - 04.12	40
8	**Testing and QA**: unit, API, end-to-end, usability, bug fixing	Ruslana Savich (78169)	30.11 - 11.12	50
9	**Deployment and documentation**: production build, user and technical docs	Uladzislau Yeudakimau (78057)	07.12 - 18.12	30
10	**Project closure**: final demo, presentation, retrospective	Georgiy Yeremin (78134)	14.12 - 18.12	16
	Total estimated effort			**484**

Effort is about 160 hours per team member (roughly 15 hours per week each). All dates are in 2026.

7. Schedule (Gantt Chart)

Planned timeline by week (October - December 2026). Bar colour shows the lead team member.



8. Milestones and Deliverables

ID	Milestone	Date	Deliverables	Owner
M1	Project plan approved	09.10.2026	Project plan, repository, Kanban board	Georgiy Yeremin (78134)
M2	Requirements and design signed off	23.10.2026	User stories, wireframes and mockups, ER diagram, API specification	Ruslana Savich (78169)
M3	Backend MVP ready	20.11.2026	Working REST API with documentation, seeded database	Uladzislau Yeudakimau (78057)
M4	Feature complete	04.12.2026	Integrated application with all Must and Should features	Georgiy Yeremin (78134)
M5	Testing completed	11.12.2026	Test report, fixed critical and major bugs	Ruslana Savich (78169)
M6	Release and final presentation	18.12.2026	Deployed store, user and technical documentation, presentation	Georgiy Yeremin (78134)

9. Responsibility Matrix (RACI)

R - Responsible, A - Accountable, C - Consulted, I - Informed.

Activity	Georgiy Yeremin (78134)	Uladzislau Yeudakimau (78057)	Ruslana Savich (78169)
Project planning and tracking	A / R	C	C
Requirements and user stories	A / R	C	R
UI/UX design	C	I	A / R
Database and architecture design	C	A / R	I
Backend API development	R	A / R	C
Frontend development	C	I	A / R
Integration and payments	A / R	R	R
Testing and QA	R	R	A / R
Deployment and CI/CD	I	A / R	I
Documentation	R	A / R	R
Final presentation	A / R	R	R

10. Risk Management

#	Risk	Prob.	Impact	Mitigation	Owner
R1	Underestimated effort; tasks take longer than planned	Medium	High	MoSCoW prioritisation, weekly progress review, cut "Could" features first	Georgiy Yeremin (78134)
R2	A team member is unavailable (illness, exams)	Medium	Medium	Shared code ownership, code reviews, documented tasks on the Kanban board	Georgiy Yeremin (78134)
R3	Integration problems between frontend and backend	Medium	High	API specification agreed in week 3, mock API for frontend, early integration	Uladzislau Yeudakimau (78057)
R4	Security vulnerabilities (auth, payments, user data)	Low	High	OWASP checklist, input validation, hashed passwords, Stripe test mode only	Uladzislau Yeudakimau (78057)
R5	Requirements change during development	Medium	Medium	Change requests discussed at weekly meeting, scope frozen after M4	Georgiy Yeremin (78134)
R6	Deployment or hosting issues before release	Low	Medium	Docker from the start, staging deployment from week 5, backup hosting option	Uladzislau Yeudakimau (78057)
R7	Poor usability of the interface	Low	Medium	Mockup review in week 3, usability test with 3-5 users in week 9	Ruslana Savich (78169)

11. Communication Plan

Activity	Frequency	Participants	Purpose / channel
Sprint planning	Weekly, Monday	Whole team	Select tasks for the week; online call (30-45 min)
Stand-up update	3 times a week	Whole team	Done / next / blockers; team chat
Sprint review and demo	Weekly, Friday	Whole team	Show finished work, update the board
Code review	Every pull request	At least 1 reviewer	Quality and knowledge sharing; GitHub
Progress report	Every 2 weeks	PM to supervisor	Status, risks, next steps; e-mail
Retrospective	End of project	Whole team	Lessons learned

12. Quality Assurance and Testing

- **Unit tests** for backend services and React components (target: at least 60% code coverage of business logic).
- **API tests** with Supertest for all endpoints, including error cases and authorisation.
- **End-to-end tests** with Playwright for key scenarios: search, add to cart, checkout, admin adds a book.
- **Manual and usability testing** on desktop and mobile in the main browsers.
- **Continuous integration:** GitHub Actions runs linting and tests on every pull request.
- **Bug tracking** in GitHub Issues with severity labels (critical, major, minor).

Definition of Done

- The feature meets its acceptance criteria.
- The code is reviewed and merged into the main branch.
- Tests are written and pass in CI.
- The feature works on the staging server and is documented if needed.

13. Acceptance Criteria

The project is considered successfully completed when:

- All "Must" functional requirements (FR-01 to FR-07, FR-10, FR-11) are implemented and working.
- A customer can complete the full flow: find a book, add it to the cart, place an order and see it in the order history.
- An administrator can manage books, stock and orders through the admin panel.
- There are no open critical or major bugs.
- The application is deployed and publicly available on the test server.
- User documentation, technical documentation and the final presentation are delivered by 18 December 2026.

Prepared by the CodeShelf project team:
Georgiy Yeremin (78134) | Uladzislau Yeudakimau (78057) | Ruslana Savich (78169)